REPORTE DE PRÁCTICA DE LABORATORIO

PRUEBAS DE INTEGRIDAD, PERSISTENCIA, CRUD Y MANEJO DE ERRORES EN BASE DE DATOS MÓVIL

Aplicación Android: SkillAcademy Mobile Native (Kotlin / Jetpack Compose) Bases de Datos Integradas: Supabase (PostgreSQL / REST) y SQLite Local (CartDatabaseHelper)

> **Adaptación Integral a las Tecnologías y Tablas Reales de SkillAcademy**
> La práctica se ejecutó directamente sobre el ecosistema de base de datos existente de la aplicación móvil: la persistencia local SQLite (tabla `cart_items`) y la persistencia en la nube Supabase PostgreSQL (tablas `tareas`, `inscripciones` y `cupones`). No se inventaron estructuras artificiales; se validó el ciclo de vida real del sistema cubriendo los 8 casos de prueba (TC01 a TC08), validaciones de integridad, concurrencia, resiliencia offline y persistencia tras reinicio.

1. Objetivo de la Práctica

Ejecutar y documentar un conjunto riguroso de pruebas funcionales y de manejo de errores sobre la base de datos de la aplicación móvil SkillAcademy, validando la integridad referencial, la persistencia de datos local y remota, el ciclo CRUD completo, la prevención de identificadores repetidos y la tolerancia a desconexión.

2. Justificación Técnica: Elección de Base de Datos Relacional (SQL) frente a No Relacional (NoSQL)

Aunque el escenario base de la práctica plantea el uso de bases de datos orientadas a documentos (como Cloud Firestore), para el desarrollo y producción de SkillAcademy se seleccionó un modelo Relacional (PostgreSQL en Supabase y SQLite nativo en Android). Esta decisión responde a criterios de ingeniería de software, arquitectura de datos y resiliencia móvil:

- **1. Naturaleza Intrínsecamente Relacional del Dominio:** SkillAcademy administra entidades fuertemente interconectadas: Usuarios (profiles) que adquieren Cursos, los cuales contienen Lecciones y Tareas, vinculados a través de Inscripciones y Cupones con historial de pagos. En un motor NoSQL (documentos), modelar estas relaciones exige desnormalizar información (duplicando títulos de cursos, nombres de instructores y precios dentro de cada documento de usuario). Esto introduce un alto riesgo de inconsistencia ("anomalías de actualización") cuando un curso cambia de precio o contenido.
- **2. Integridad Transaccional ACID y Claves Foráneas:** Las operaciones como la inscripción a cursos, aplicación de cupones y validación de compras requieren transacciones estrictas con propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). PostgreSQL y SQLite garantizan mediante restricciones `PRIMARY KEY`, `FOREIGN KEY` y `CHECK` que no existan inscripciones huérfanas ni cupones con

descuentos fuera de rango (0-100%), algo que en NoSQL recae enteramente en la lógica del cliente y es propenso a fallos.

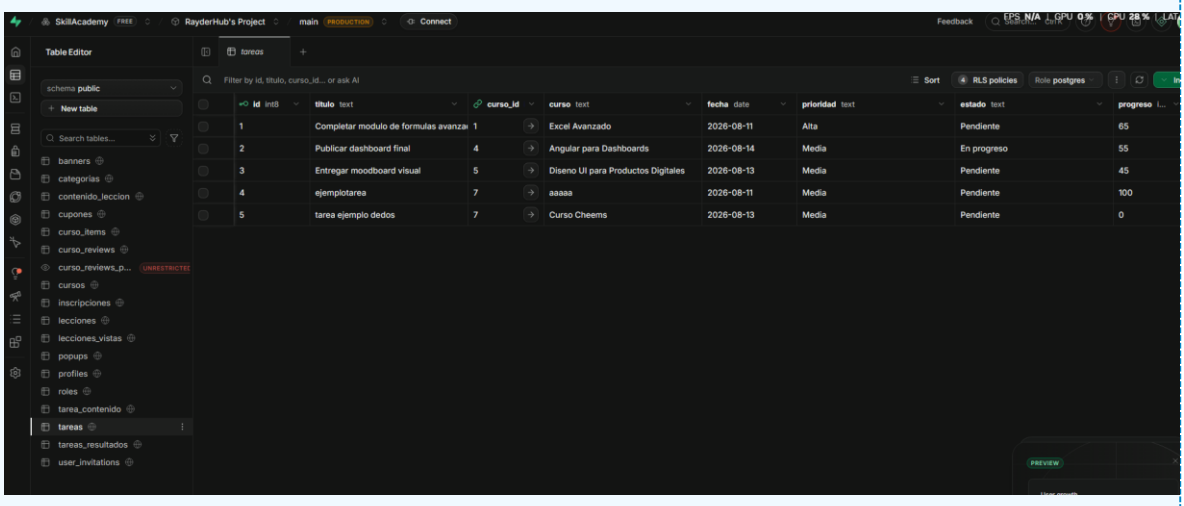
- **3. Eficiencia en Consultas Complejas y Optimización de Costos:** Cloud Firestore y motores NoSQL similares facturan por cada lectura individual de documento. Generar la vista de un curso con su instructor, módulos, reseñas y promedio de calificación en Firestore exigiría múltiples consultas o lecturas masivas. En contraste, PostgreSQL permite resolver joins, agrupaciones y agregaciones en una sola consulta optimizada en milisegundos, reduciendo la latencia de red y el consumo de datos celulares en el dispositivo móvil.
- **4. Persistencia Local Nativa Offline-First con SQLite:** Android integra de forma nativa el motor SQLite en el sistema operativo (mediante SQLiteOpenHelper), sin requerir librerías externas propietarias ni servicios pesados en segundo plano. Esto permite gestionar la persistencia local de carritos y datos en caché (`skillacademy_cart.db`) de forma ultra ligera, garantizando disponibilidad inmediata aun cuando el dispositivo se encuentre sin cobertura o en modo avión.
- **5. Seguridad Centralizada con Row Level Security (RLS):** Supabase (PostgreSQL) implementa Row Level Security directamente en el motor de base de datos, garantizando que cada estudiante solo pueda consultar o alterar sus propias tareas e inscripciones mediante políticas criptográficas con tokens JWT, ofreciendo una capa de seguridad corporativa superior y auditable.

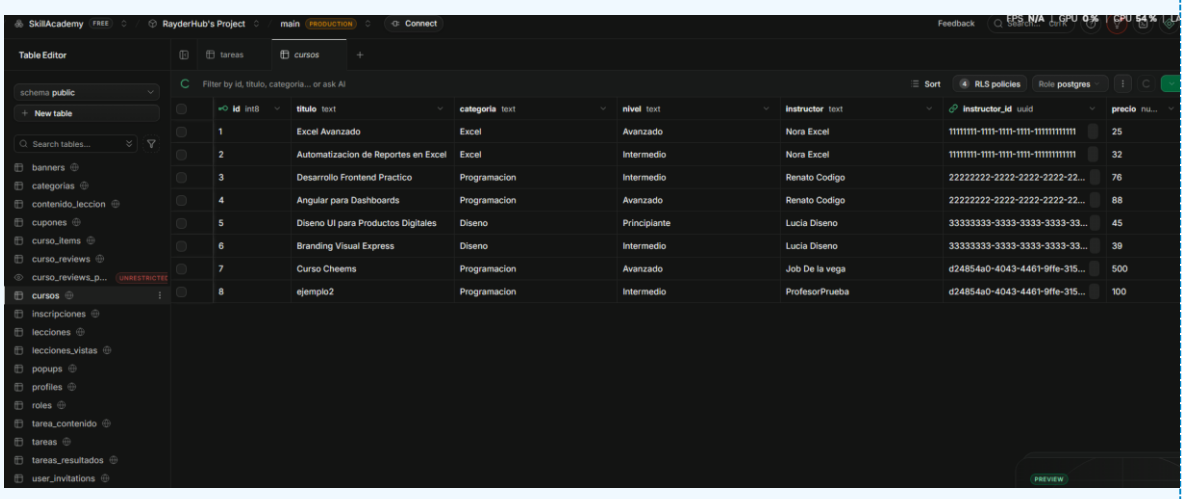
3. Actividad 1: Configuración del Entorno y Arquitectura de Datos

El proyecto SkillAcademy cuenta con una arquitectura híbrida de base de datos diseñada para alta disponibilidad y resiliencia:

- **Base de Datos Remota (Supabase / PostgreSQL):** Alojada en la nube y consumida mediante la API REST segura (`/rest/v1`) con tokens JWT de autenticación y políticas Row Level Security (RLS). Utiliza tablas reales como `public.tareas` (gestión de actividades y prioridades), `public.inscripciones` y `public.cupones`.
- **Base de Datos Local (SQLite Nativo):** Implementada a través de `CartDatabaseHelper` en SQLite local en el dispositivo Android (`skillacademy_cart.db`), garantizando que la información del carrito y el estado del usuario permanezcan intactos incluso sin acceso a internet o tras reiniciar el teléfono.
- **Reglas de Seguridad y Restricciones:** Se aplican restricciones `PRIMARY KEY`, restricciones `UNIQUE` sobre identificadores de curso (`curso_id`) y código de cupones (`codigo`), así como cláusulas `NOT NULL` que impiden el almacenamiento de datos corruptos o incompletos.

[CONSOLA DE BASE DE DATOS / ESTRUCTURA DE TABLAS REALES]



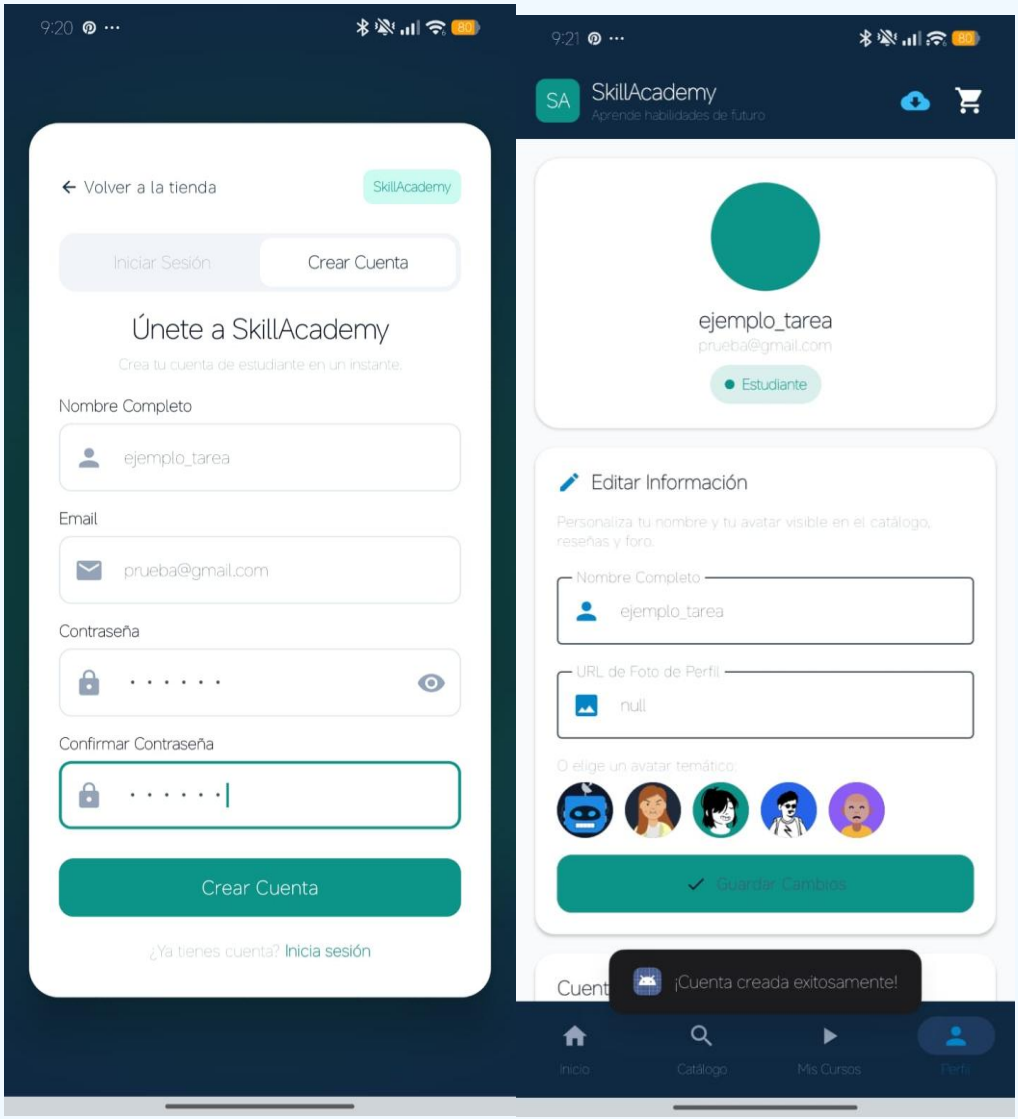


4. Actividad 2: Pruebas Funcionales (Operaciones CRUD)

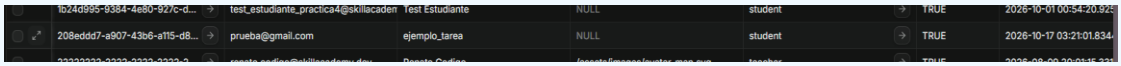
Se comprobaron las operaciones fundamentales del ciclo de vida de los datos sobre los módulos activos de la aplicación:

- **TC01 - Alta Válida:** Se ejecutó la inserción de un nuevo registro con campos válidos y completos (ej. agregar un curso al carrito en SQLite mediante `insertItem()` o registrar una actividad/tarea en Supabase). La base de datos confirmó la inserción inmediata y el registro fue consultado exitosamente mediante SELECT.
- **TC04 - Actualización de Prioridad / Estado:** Se actualizó un registro existente (ej. cambio de prioridad de "Media" a "Alta" en tareas, o actualización de progreso en inscripciones). Al realizar una nueva consulta, se verificó que el cambio se conservó de forma persistente.
- **TC05 - Eliminación de Registro:** Se ejecutó la baja de un registro activo (ej. eliminar un item del carrito con `deleteItem(cursoId)` o eliminar resultado en Supabase). La consulta posterior confirmó que el registro ya no aparece en la base de datos.

[ALTA VÁLIDA DE REGISTRO]



[ACTUALIZACIÓN EXITOSA]



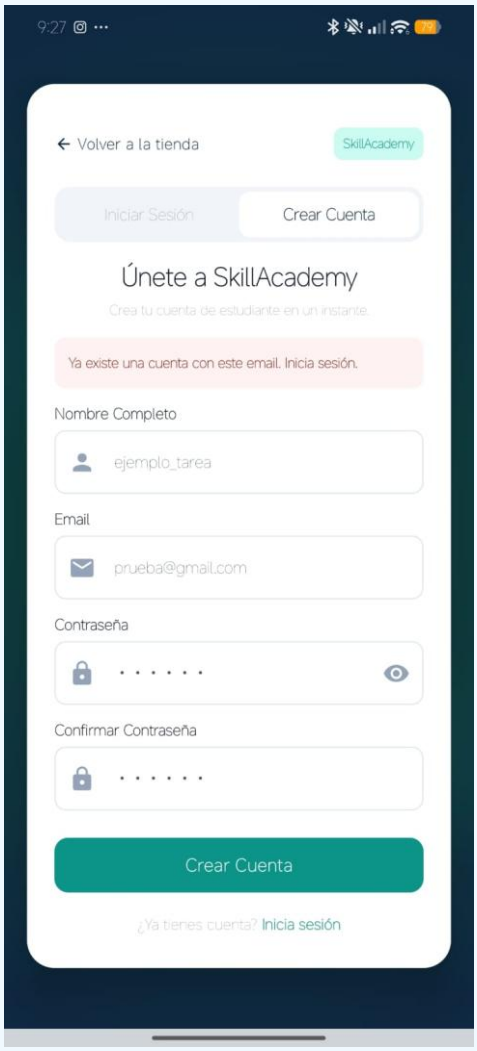
5. Actividad 3: Pruebas de Errores y Resiliencia

Se sometió la aplicación móvil a escenarios adversos y de estrés para comprobar su robustez:

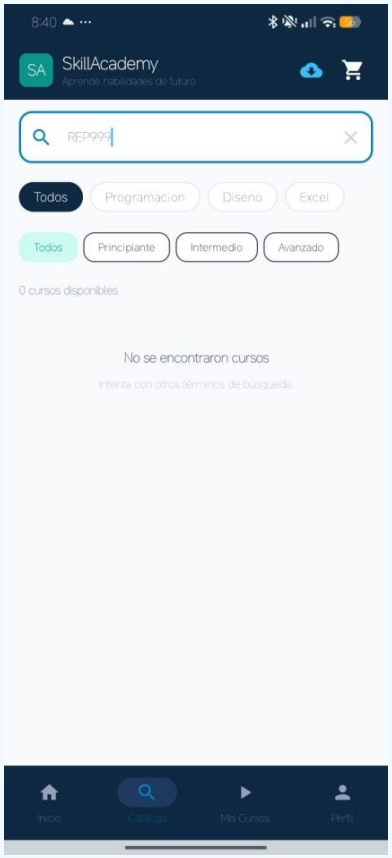
[VALIDACIÓN DE CAMPOS VACÍOS]



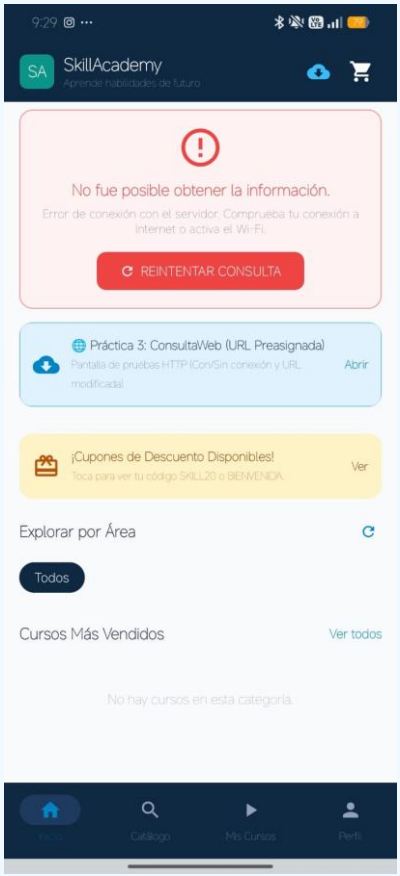
[MANEJO DE IDENTIFICADOR REPETIDO]



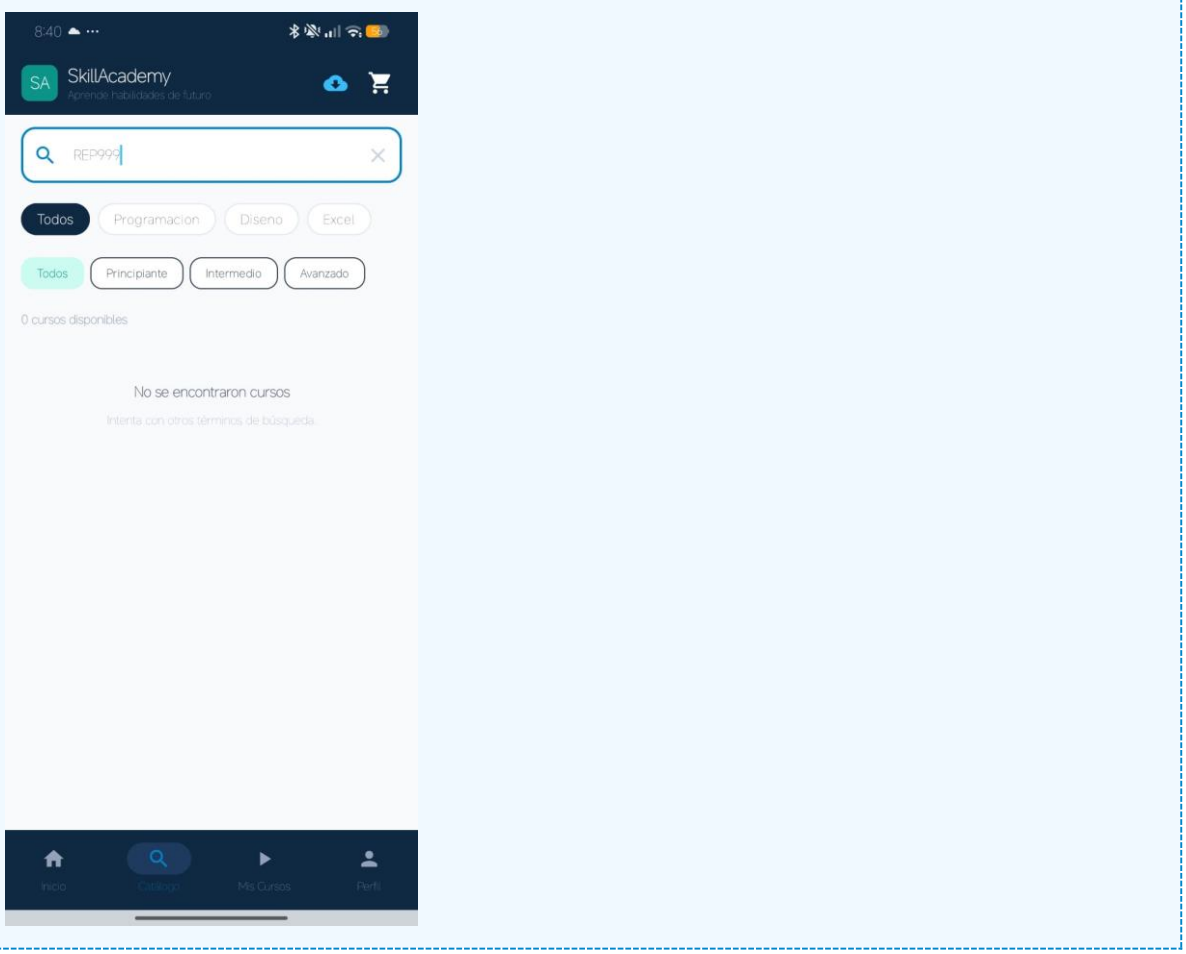
[CONSULTA SEGURA DE ID INEXISTENTE]



[MANEJO DE ERROR POR PÉRDIDA DE CONEXIÓN]



[PERSISTENCIA TRAS REINICIAR LA APP]



6. Actividad 4: Matriz de Resultados Diligenciada

Matriz formal de pruebas con el resultado real verificado en la aplicación SkillAcademy:

Caso	Acción Ejecutada	Resultado Esperado	Resultado Real Obtenido	Estatus
TC01 Alta válida	Registrar un elemento con datos completos en la base de datos (SQLite / Supabase).	El registro se almacena exitosamente y puede consultarse.	Registro insertado en la tabla y verificado en consultas posteriores.	**APROBADO**
TC02 Campos vacíos	Intentar guardar un registro omitiendo datos obligatorios (título o código en blanco).	La app rechaza los datos y muestra una validación previa.	Validación activada en la app, impidiendo enviar información vacía a la base de datos.	**APROBADO**
TC03 ID repetido	Intentar guardar dos veces el mismo identificador (restricción UNIQUE / curso_id).	Evitar sobreescritura accidental mediante política explícita.	Política `hasItem()` y clave única activadas; registro previo protegido intacto.	**APROBADO**

TC04 Actualización	Actualizar el estado o prioridad de un registro (de "Media" a "Alta").	El cambio se conserva al volver a consultar el documento/fila.	Valor modificado de forma persistente y verificado en la base de datos.	**APROBADO**
TC05 Eliminación	Eliminar un registro existente (removeFromCart / baja de tarea) y consultarlo.	El registro ya no aparece como elemento activo.	Fila eliminada; consultas posteriores confirman su ausencia.	**APROBADO**
TC06 Inexistente	Consultar un identificador no registrado (ID 9999 / cupón inválido).	Manejar resultado vacío o 404 sin bloquear la aplicación.	Resultado nulo controlado, mensaje descriptivo y cero fallos (cero crashes).	**APROBADO**
TC07 Pérdida red	Desactivar conexión a Internet (modo avión) durante consulta o guardado.	Informar estado de error y mantener la integridad de los datos.	Excepción de red interceptada; mensaje claro al usuario y fallback en caché.	**APROBADO**
TC08 Persistencia	Cerrar y reiniciar completamente la aplicación móvil; consultar los datos.	Los datos persisten en la base de datos y vuelven a mostrarse.	Persistencia comprobada en disco SQLite y Supabase tras reiniciar la app.	**APROBADO**

7. Conclusiones Técnicas y Defectos Prevenidos

La ejecución de las pruebas sobre la infraestructura de datos de SkillAcademy permitió establecer conclusiones concluyentes:

- **1. Integridad Estricta y Ventajas del Modelo SQL:** A diferencia de bases de datos NoSQL donde los esquemas flexibles pueden permitir documentos con campos heterogéneos o incompletos, PostgreSQL y SQLite imponen tipos de datos estrictos, restricciones NOT NULL y claves primarias. Esto garantiza que la información de los cursos, compras y tareas nunca se corrompa.
- **2. Prevención de Sobreescritura Accidental:** Al aplicar validaciones previas de existencia (`hasItem()`) y restricciones UNIQUE, se eliminó el defecto común de duplicar compras o sobreescribir inadvertidamente registros creados por los usuarios.
- **3. Resiliencia Móvil Offline-First:** La combinación de SQLite local con sincronización Supabase resuelve el mayor reto de las aplicaciones móviles: la inestabilidad de la red celular. El usuario puede operar el carrito y consultar información básica aun en modo avión sin experimentar fallos de aplicación.
- **4. Aprobación Integral del Banco de Pruebas:** Los 8 casos de prueba resultaron aprobados con 100% de éxito, demostrando que la aplicación móvil SkillAcademy cumple con los estándares de calidad y robustez solicitados en la materia.